matches, it is better to look for names that start with the search terms. All the queries you've performed so far with the `.filter()` method have had their clauses AND-ed together. To search for a first or last name beginning with a search term, you need clauses OR-ed together.

In Django, you can create more complex queries by using the `Q` object, found in the `django.db.models` module. `Q` is short for *query*, and you build the objects using the same arguments as you pass to `.filter()`. You can combine `Q` objects together to compound them, using Python's binary operators, like AND (`&`) and OR (`|`). This allows you to form arbitrarily complex queries, and then submit the resulting `Q` object as a single argument to a `.filter()` call. In chapter 11, when you passed `q=name__icontains` into a `Field` class to modify a related-object query, it was using this same technique under the covers.

In addition to searching using both first and last name, it would be nice to allow the user to enter multiple search terms. To support this, the code needs to parse the text based on whitespace, use each term in two `Q` objects for the first and last names, and then combine all the `Q` objects with an OR (`|`). The finished query searches for instances of any of the search terms in either name. The following listing contains the first attempt at a search view.

Listing 12.6 A search view for musicians by their names

```
# RiffMates/bands/views.py
...
from django.db.models import Q
...

def search_musicians(request):
    search_text = request.GET.get("search_text", "")
    search_text = urllib.parse.unquote(search_text)
    search_text = search_text.strip()

    musicians = []

    if search_text:
        parts = search_text.split()

        q = Q(first_name__istartswith=parts[0]) | \
            Q(last_name__istartswith=parts[0])
        for part in parts[1:]:
            q |= Q(first_name__istartswith=part) | \
                 Q(last_name__istartswith=part)

        musicians = Musician.objects.filter(q)

    data = {
        "search_text": search_text,
        "musicians": musicians,
    }

    return render(request, "search_musicians.html", data)
```

Retrieve the search terms from the query parameter.

A URL parameter gets encoded. Turn it back into a regular string.

Split the search terms into a list.

Build Q objects OR-ed together to search for this term in both the first and last name fields of the Musician object.

Loop through subsequent search terms.

OR another set of Q objects to the combined query.

Perform the database search by calling .filter() with the combined Q objects.